

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5

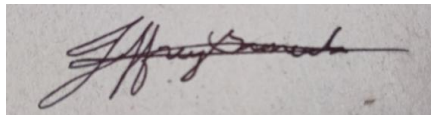
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I E.Jeffrey Samuels certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

ANSWERS – INDUSTRY PROBLEM SOLVING TASK

Q1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Recommended strategy: B+ Tree indexed file organization.

- A B+ Tree is suitable because product catalogs need fast exact searches (for example, by ProductID) and efficient range queries (such as price or category ranges).
- The data records can be stored in data pages while a B+ Tree index is built on ProductID and additional secondary indexes can be created on frequently searched attributes such as Category or Price.
- With 10 million records, a linear scan in a heap file may require checking a very large portion of the data. A B+ Tree reduces point-search work to approximately $O(\log N)$ index traversal.
- Storage cost is higher than a pure heap file because index pages must also be maintained. Insertions and deletions also incur index-maintenance cost, but the improvement in search performance is substantial for a high-volume e-commerce workload.

Factor	Heap File	B+ Tree Indexed
Point search	$O(N)$	$O(\log N)$
Range search	$O(N)$	$O(\log N + k)$
Insert	$O(1)$ average	$O(\log N)$
Storage	Lowest	Higher due to index
Best use	Heavy sequential inserts	Fast product retrieval

Cost analysis: The main trade-off is extra index storage and update overhead versus much lower search cost. For 10 million records, the B+ Tree approach is justified because retrieval speed is usually more valuable than the additional index storage in an e-commerce system.

Q2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Recommended strategy: Composite B+ Tree index on (AccountNumber, TransactionDate).

- Use a B+ Tree because it supports both equality search and ordered range search.
- The composite key (AccountNumber, TransactionDate) first groups transactions by account and then keeps each account's transactions ordered by date.
- For an equality-plus-range query such as AccountNumber = A and TransactionDate BETWEEN D1 AND D2, the index can locate the account quickly and scan only the required date interval.
- A separate index on TransactionDate is optional if the system also performs many date-only queries across all accounts.

Example index: CREATE INDEX idx_account_date ON Transactions(AccountNumber, TransactionDate);

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Recommended solution: Primary B+ Tree index + Secondary B+ Tree index.

- Primary index: Build a B+ Tree on PatientID. Since PatientID is unique, each key points to one patient record (or record/page location).
- Secondary index: Build a B+ Tree on DoctorName. Because many patients can be associated with the same doctor, each doctor-name entry points to a list of matching record locations.
- This gives direct access through PatientID and efficient retrieval of all patients associated with a doctor.
- The secondary index is maintained separately, so updates to patient information can be reflected without changing the primary ordering.

Access path	Index	Result
PatientID	Primary B+ Tree	Single patient record
DoctorName	Secondary B+ Tree	All matching patient records

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Recommendation: B+ Tree.

Feature	B-Tree	B+ Tree
Search	$O(\log N)$	$O(\log N)$
Insertion	$O(\log N)$	$O(\log N)$
Deletion	$O(\log N)$	$O(\log N)$
Range queries	Good	Excellent
Leaf-node links	No	Yes
Sequential access	Less efficient	Highly efficient

- Both structures support dynamic insertion and deletion, but B+ Tree keeps all record pointers at the leaf level.
- Linked leaf nodes make it easy to scan shipments over a range such as a date interval, warehouse, or shipment ID range.
- B+ Trees generally have higher fan-out because internal nodes store only keys and child pointers, reducing tree height and disk I/O.

Conclusion: B+ Tree is more suitable for a logistics database because it combines efficient dynamic updates with excellent range and sequential access.

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Proposed scheme: Hash EmployeeID to a bucket number.

- Example hash function: $h(\text{EmployeeID}) = \text{EmployeeID} \bmod 101$, giving 101 primary buckets.
- Choose the number of buckets based on expected records per bucket and leave some free capacity to reduce overflow.
- Static hashing keeps 101 buckets fixed. It is simple and inexpensive when the employee count is stable, but growth can create overflow chains.
- Extendible hashing uses a directory and dynamically splits buckets when they become full. It therefore handles future growth better.

Factor	Static Hashing	Extendible Hashing
Implementation	Simple	More complex
Growth	Weak	Excellent
Overflow	Higher risk	Lower risk
Maintenance	Low	Higher
Recommendation	Only if size is stable	Preferred for future growth

Conclusion: For a 5000-employee HR system, static hashing is acceptable if the size will remain nearly fixed. Extendible hashing is the safer long-term design because employee records can grow without requiring a complete reorganization.

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Recommended combined structure: Hash index on FlightNumber + B+ Tree index on FlightDate.

- Hash index: best for equality/point lookup such as $\text{FlightNumber} = \text{AI202}$.
- B+ Tree index: best for ordered date intervals such as $\text{FlightDate BETWEEN D1 AND D2}$.
- The two indexes can coexist on the same reservation data because they serve different access patterns.
- A composite B+ Tree on (FlightNumber, FlightDate) may be added if combined flight-and-date queries are frequent.

Query type	Recommended index	Reason
$\text{FlightNumber} = \text{value}$	Hash	Very fast equality lookup
$\text{Date BETWEEN two dates}$	B+ Tree	Efficient ordered range scan
$\text{FlightNumber} + \text{Date}$	Composite B+ Tree (optional)	Supports combined predicate

Q7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Recommended design: Heap file + secondary B+ Tree indexes.

- Store student records in a heap file because new student records can be inserted without preserving a strict physical order.
- Secondary index 1: B+ Tree on Department for fast equality searches such as Department = CSE.
- Secondary index 2: B+ Tree on CGPA for range queries such as CGPA BETWEEN 8.0 AND 9.0.
- The CGPA index is particularly effective because B+ Tree leaves are ordered, allowing a sequential scan of only the matching range.

Requirement	Design	Benefit
Record storage	Heap file	Fast, simple insertion
Department search	B+ Tree secondary index	Fast equality lookup
CGPA range	B+ Tree secondary index	Efficient range scan

Q8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

- Block size = 4 KB = 4096 bytes.
- Record size = 50 bytes.
- Records per block = $\text{floor}(4096 / 50) = 81$ records.
- Number of data blocks = $\text{ceil}(100000 / 81) = 1235$ blocks.
- The last block contains $100000 - (1234 \times 81) = 46$ records, so $46 \times 50 = 2300$ bytes are used and $4096 - 2300 = 1796$ bytes remain unused in the final block.

Organization	Approx. data blocks	Utilization characteristics
Heap	1235	Good packing; order not maintained
Sorted	1235	Similar packing; extra maintenance for order
Hashed	1235+	Similar ideal packing, but overflow blocks may increase storage

Final observation: Ignoring headers, tombstones, and overflow overhead, all three require about 1235 data blocks. Sorted and hashed files may need extra space for indexing or overflow structures.

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Recommended strategy: composite B+ Tree on (user_id, timestamp) plus a separate hashtag index.

- Primary composite index: (user_id, timestamp). This supports “all posts by a user” and “a user’s posts between two timestamps”.
- Hashtag index: maintain an index from hashtag to post IDs. A B+ Tree is suitable for ordered hashtag keys; an inverted mapping is useful when one hashtag matches many posts.
- Timestamp index: add a separate B+ Tree on timestamp only if global time-range queries are common across all users.
- At 500 million posts, the indexes should be partitioned or distributed to control tree size, memory usage, and maintenance cost.

Query	Suggested index
Posts for one user	B+ Tree (user_id, timestamp)
One user in a time range	B+ Tree (user_id, timestamp)
Posts for a hashtag	Hashtag → PostID index
Global time range	B+ Tree (timestamp), if required

Q10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$ average	$O(N)$ in worst case for physical order	$O(1)$ average
Delete	$O(N)$ search unless address known	$O(N)$ worst case / reorganization	$O(1)$ average after locating bucket
Exact search	$O(N)$	$O(\log N)$	$O(1)$ average
Range search	$O(N)$	$O(\log N + k)$	Poor / requires many buckets
Best feature	Fast simple insert	Excellent ordered/range access	Excellent exact-key access

- Heap File: suitable for very frequent inserts but exact searching across 1 million transactions can require a large scan.
- Sorted File: excellent when reports frequently request transaction ranges by date or transaction ID, but maintaining order makes insertion expensive.

- Hashed File: best for exact lookup by a transaction key and provides average $O(1)$ insertion/search, assuming a good hash function and controlled overflow.
- For a billing system dominated by exact transaction retrieval, choose hashing as the primary organization. Add a B+ Tree secondary index when range/date reporting is important.

Overall conclusion: For 1 million daily transactions, Hashed File is the best primary choice for high-speed exact lookups and frequent inserts. A secondary B+ Tree index can cover date-range queries.